



Macgear 3PL

Logistics Portal

How the 3PL Portal Works

Internal reference — architecture, processes, billing logic, and the key decisions & assumptions behind the customer visibility portal.

Audience: Macgear internal staff (ops, finance, warehouse) · **Status:** live build; first customer Mova, stock expected ~end Jul 2026 · **Not for customer distribution.**

1. What the portal is for

Macgear's 3PL service receives, stores, and dispatches stock it **does not own**, on behalf of customers, and charges handling & storage fees. The portal does two jobs on top of NetSuite:

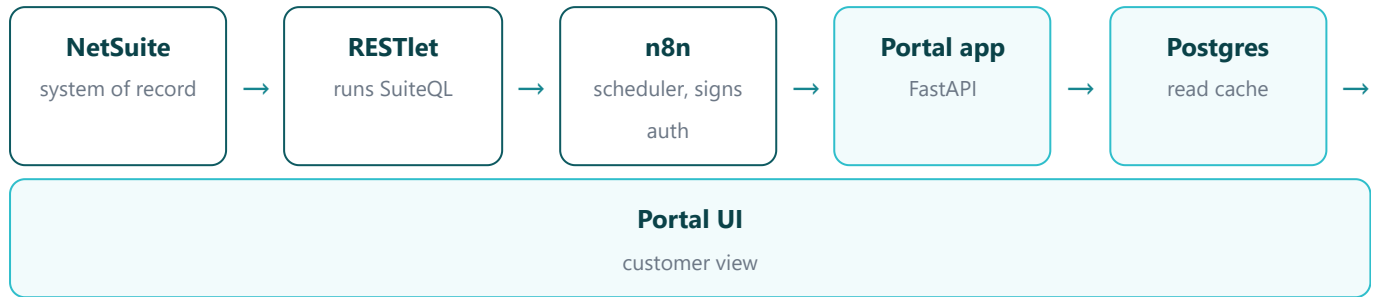
- **Visibility** — gives each customer a self-service, near-live view of their stock with Macgear (6 views: stock on order, receipts, stock on hand, fulfilments, invoices, rate card). This is the priority.
- **Billing automation** — replaces the manual weekly saved-search process with an automated draft-invoice run for the 5 service charges.

It is **multi-tenant** from day one — Mova is the flagship, Skriva is a smaller live reference, and more customers are expected. A new customer is config, not a code change.

2. Architecture & data flow

Macgear 3PL Portal — Internal reference · Confidential · Not for customer distribution

Four moving parts. The golden rule: **NetSuite is the system of record; the portal app never holds NetSuite credentials and never calls NetSuite directly.**



- **RESTlet** (SuiteScript in NetSuite) is the *only* thing that talks to NetSuite for the portal. It runs validated SuiteQL reads and creates draft invoices.
- **n8n** (on the droplet) is the scheduler. It signs NetSuite Token-Based Auth, calls the RESTlet, and POSTs the rows to the app's token-authenticated `/admin/ingest` endpoint. It holds the secrets.
- **App + Postgres** — the app upserts rows into a Postgres **read cache** and serves the portal. It makes no outbound NetSuite calls and runs no AI/MCP at runtime.

WHY THIS SHAPE

Same pattern as the existing vendor-credit-claims app (Docker on the n8n droplet). Keeping NetSuite credentials out of the customer-facing app is a security boundary; routing everything through one RESTlet keeps the NetSuite surface area tiny and auditable.

Billing write-back (the one exception)

The portal never writes operational stock data back. The **only** write-back is draft invoices: clicking “**Queue for NetSuite**” marks a billing run ready → n8n polls `/admin/billing/pending` → the RESTlet creates a **DRAFT** invoice for approval → the new invoice id is reported back → the next read-sync pulls the real invoice (status, edits, payments) back into the portal.

3. The six visibility views

Macgear Internal — Internal reference · Confidential · Not for customer distribution

Each view is derived from the cache, which mirrors the shape of the underlying NetSuite transactions. A customer's stock is isolated by their NetSuite **classification** ("**brand**" **class**) and **location**.

#	VIEW	NETSUITE SOURCE	HOW IT'S SCOPED / NOTES
1	Stock on order	Open Purchase Orders	Vendor = customer's supplier + item class. Only lines not yet fully received. Fully-received POs are pruned (see Assumptions).
2	Item receipts	Item Receipts	Scoped by item class. Shows the source PO # for each receipt.
3	Stock on hand	Inventory balance	Scoped by location AND class . Near-live (15-min refresh) with a "● live · updated N ago" stamp.
4	Item fulfilments	Item Fulfilments	Both SO (shipped to customer) and VRMA (returned to vendor). Source is decided by the fulfilment's entity.
5	Invoices	Customer invoices	The customer's invoices, drillable to per-charge line detail.
6	Rate card	Config (not NetSuite)	Per-customer, effective-dated charges.

4. How stock moves — receiving & dispatch

Receiving (inbound)

A **\$0 PO** on the supplier account against the 3PL location → an inbound shipment per container → an item receipt on arrival. The receipt drives the putaway charge.

Dispatch (outbound) — the standard procedure

Two NetSuite paths move stock out. They record **different economic events**, so pick by reality, not convenience. The deciding question: **is Macgear taking ownership of (paying for) the stock?**

ANSWER	PROCEDURE	EFFECT
Yes — Macgear buys it DEFAULT / majority case	VRMA on the vendor (customer-as-supplier) account, then a normal-price buy-in PO into the regular warehouse	The \$0 VRMA clears the 3PL receipt and removes it from the customer's held inventory; the PO brings it onto Macgear's books as owned stock at cost. Fulfilment source = VRMA .
No — dispatch on the customer's behalf	\$0 Sales Order on the customer account	Pure logistics; ownership stays with the customer. Fulfilment source = SO .

FAILURE MODE TO AVOID

Using an **SO when Macgear is actually purchasing** ships stock off the customer's 3PL holding but brings nothing onto Macgear's books → **phantom inventory + unrecognised COGS**. When Macgear is buying, always use VRMA + buy-in PO. Billing is identical either way (\$1.00/unit pick), so always choose by the accounting reality.

5. Billing — the five charges

Internal reference · Confidential · Not for customer distribution

The billing engine reads the cache for a period and produces one line per charge from the customer's effective-dated rate card. Figures shown are Mova's rates.

CHARGE	RATE	BASIS	DERIVED FROM
Container unload	\$1,500	per container	Count of inbound shipments received in the period
Putaway	\$1.00	per unit	Sum of item-receipt units in the period
Storage	\$4.50	per pallet / week	Average daily pallets × weeks in the period. Pallets = $\text{ceil}(\text{units on hand} \div \text{units per pallet})$.
Picking — SO	\$1.00	per unit	Units on SO fulfilments (shipped to customer)
Picking — VRMA	\$1.00	per unit	Units on VRMA fulfilments (returned to vendor)

STORAGE IS THE SUBTLE ONE

Because stock on hand is now snapshotted every ~15 minutes (one row kept per day), the engine **averages the daily pallet totals across the week** rather than summing every snapshot — summing would over-charge roughly 7×. With a single snapshot in a period it degrades to “that reading held all week.”

Billing run lifecycle

`draft` → `ready_to_push` → `pushed` → `invoiced`. A period already queued/pushed/invoiced is locked from being re-billed. Rate changes write a **new effective-dated rate card**, so re-running an old period reprices correctly against the rate that applied then.

6. Sync cadence — two lanes

Both lanes feed the same n8n code node; a “mode” flag decides what runs.

FAST LANE — EVERY 15 MIN

Stock on hand only. No billing writes. Keeps the SOH view near-live (“● live · updated N ago”) without re-pulling everything 96×/day. NetSuite inventory balance updates in real time, so 15-min polling sees genuine change.

FULL LANE — DAILY + WEEKLY

All six entities plus the billing draft-invoice push. Runs daily, with one run in the weekly billing window. Defaults to “full” if no mode is set, so existing wiring keeps working.

SOH detail: today's row is overwritten in place each fast-lane run; older days persist as daily history that the storage charge averages over.

7. Major assumptions & decisions (the things to know)

“BRAND” = A NETSUITE CLASSIFICATION, NOT TEXT

Each customer's stock is tagged with a NetSuite **class** (e.g. a Mova class) and lives at a **location**. Scoping is by **class** primarily because the item record reliably carries it, whereas manually-keyed transaction lines often don't. Skriva proved you can't rely on a dedicated location alone (it shares the Auckland warehouse), which is why class is the anchor.

CONFIG MUST MATCH WHERE STOCK ACTUALLY LANDS

SOH is filtered by **location AND class**, but receipts are filtered by **class only**. So if a customer's configured location id doesn't match where stock is physically received, **receipts will show but stock on hand stays empty** — a config mismatch, not a bug. (This was hit and confirmed in testing.) Likewise, a VRMA only appears if its vendor equals the customer's configured supplier id.

“REPLACE” SEMANTICS FOR SNAPSHOTS

The SOH pull and the open-PO pull are treated as the **complete current picture**. Anything absent from a pull is reconciled, not left stale:

- **Stock on hand:** an item that drops to zero disappears from NetSuite's inventory balance, so any item missing from the pull is zeroed — otherwise a shipped-out SKU would show its last quantity forever.
- **Stock on order:** a fully-received PO drops out of the open-PO query entirely, so any cached PO not in the pull is pruned — otherwise it would linger showing stale outstanding quantity.

TWO NETSUITE QUERY QUIRKS WORTH REMEMBERING

- **Source PO on a receipt:** NetSuite won't let SuiteQL select `createdfrom`, so the receipt's PO number is found by walking the transaction-line link table back to the originating Purchase Order.
- **VRMA fulfilments:** a VRMA emits a single **negative** inventory line with no positive counterpart, while a customer SO emits a +/- pair. Picking units are therefore counted off the inventory **ASSET** line using its absolute value — this captures VRMAs and de-duplicates SO pairs. (An earlier “count positives only” rule silently dropped every VRMA even though stock left NetSuite.)

OPERATIONAL NICETIES

No database migration tool (Alembic) is used — the app runs a small idempotent “add column if missing” step at startup, so deploys self-heal. The UI also has dark mode and a clickable logo that returns to the overview.

8. Access, roles & multi-tenancy

Macgear 3PL Portal — Internal reference - Confidential - Not for customer distribution

ROLE	CAN SEE / DO
admin	Everything, plus the admin console — manage users and per-customer rate cards.
internal	All customers + run billing. No user/rate-card admin.
customer	Locked to their own customer; visibility views only — no billing run.

Login is per-user (email + hashed password, signed-cookie session). View permissions default by role and can be overridden per user. Every cache row is keyed by customer, so customers can never see each other's data.

9. Deployment & operations (quick reference)

- **App:** Docker on the n8n droplet. Update with `git pull && docker compose up -d --build`. The only required env is `SYNC_TOKEN` (plus `DATABASE_URL`).
- **RESTlet:** when its SuiteQL changes, the script must be re-uploaded in NetSuite, then a full sync run for the change to take effect (cached rows refresh on next pull).
- **n8n:** holds the NetSuite auth secrets, the customer config, and the charge-type → NetSuite item map used when creating invoices. Customers are added via the admin console — the next sync picks them up.
- **To onboard a new customer:** add them in the admin console with their NetSuite ids (customer, supplier, location, class) and a rate card. No code or schema change.

ONE-LINE SUMMARY

NetSuite stays the source of truth; a single RESTlet + n8n feed a Postgres read-cache on a 15-minute (stock) / daily (everything) cadence; the FastAPI portal serves customers a live view and turns the cached data into draft invoices for the five 3PL charges — with stock isolated per customer by NetSuite class & location.